Using Software Agents in a Distributed Computing System for Procedural Planetoid Terrain Generation

Yevheniya Levus
Software Department
Lviv Polytechnic National University
Lviv, Ukraine
yevheniia.v.levus@lpnu.ua

Rüdiger Westermann
Department of Informatics
Technical University of Munich
Munich, Germany
westermann@tum.de

Mykola Morozov
Department of Informatics
Technical University of Munich
Munich, Germany
mykola.morozov@tum.de

Roman Moravskyi
Software Department
Lviv Polytechnic National University
Lviv, Ukraine
roman.moravskyi.mnpzm.2021@lpnu.ua

Pavlo Pustelnyk
Software Department
Lviv Polytechnic National University
Lviv, Ukraine
pavlo.pustelnyk.mnpzm.2021@lpnu.ua

2022 IEEE 17th International Conference on Computer Sciences and Information Technologies (CSIT) | 979-8-3503-3431-9/22/$31.00 ©2022 IEEE | DOI: 10.1109/CSIT56902.2022.10000868

Abstract – **Procedural terrain generation is becoming more widespread with numerous applications in different domains, like video games, movies, and simulations. As the need for content size, quantity, and fidelity rise, old methods become insufficient to perform the task. The planetary scale also imposes new limitations on data continuity and uniqueness. This paper proposes a novel cloud-based planetoid terrain generation system based on software agents. The performance is also improved through vertical parallelism using Morsels and horizontal parallelism using multithreading. The workers processing generation requests are spread per cloud computation instance, so they can be further scaled as the number of requests increases.**

Keyword – software agents, distributed computing, planetoids, terrain generation.

I. INTRODUCTION

Nowadays, the digital content creation industry depends on the rapid creation of plausible large-scale computer-generated terrain, which has become more prevalent in recent years, and the related tools have undergone substantial scientific and technological progress. The widespread use of virtual worlds can be witnessed in cinematography and video games, which lead to the use of planetoids to generate, store, and display these large landscapes. Not only are computer-generated planetoids used for entertainment, but also for science and education. For instance, simulation of different natural processes in three-dimensional virtual worlds [1] can be used for study and analysis.

Classical computer generation approaches were predominantly based on manual editing and basic algorithms, making them extremely difficult to automate or achieve realistic results. On the contrary, any modern application in this domain is designed to generate and visualize large virtual worlds requiring significant data processing, computational power, and process automation to achieve the most plausible computer-generated worlds generation and visualization with the least manual intervention.

Many existing works in this domain approach this task using different methods. Out of such methods, noise algorithms [2] stand out as the most commonly used for the procedural generation of heightmaps. While other methods allow more control over the terrain features like subdivision methods [3] or Fourier synthesis methods [4], noise algorithms have an obvious advantage in generating near-infinite seamless terrains. There are also works [5, 6] dedicated to the research of machine learning usage in terrain synthesis that aims to solve the problem of repetitive terrain output. While these works generally solve the procedural terrain generation problem, they lack scalability, extendibility, or more refined control over the generation process steps. Moreover, they cannot be easily modified to incorporate into a system designed to generate planetoids procedurally directly.

To tackle the problem of combining various algorithms, scientists proposed a pipelined approach using software agents [7, 8] to generate virtual worlds. Each agent represents some part of the generation phase and can be opted in or out of this process. Using software agents helps introduce the system's extendibility as new agents can be added to generate new features or enhance the detail of the previously synthesized terrain. A method to flexibly manage quantitative and qualitative indicators of landscape modeling for spherical surfaces was proposed in the work [9]. In contrast to the previously considered works, this project aims to develop a virtual planetoids synthesis framework that will use a software agents approach and focus on performance and versatility.

II. ANALYSIS AND PROBLEM STATEMENT

Although the virtual terrain generation process, in general, has reached a good level of detail and realism, it still is distributed between many specialized research domains.

The work by C. J. Castle and A. T. Crooks [10], among others, proposes to use procedural generation of GIS data with a fixed entity layout. While this approach produces accurate results, it is challenging to integrate such a solution into larger systems without bringing the software dependencies along.

The situation becomes even more radical in the field of Machine Learning. There is ongoing research not only for the generation but also for incorporating these methods in existing systems [5, 6], including data conversion and visualization.

To unify the process, software agents have been proposed as a viable solution spanning research domains with real-world applications ranging from digital image processing [11] to geodesical exploration [12].

While their principles were outlined in prior research [13], even taking dependencies into account by producing "software graphs", only recently has there been research directed towards multimedia content generation, including games and movies [14].

979-8-3503-3431-9/22/$31.00 ©2022 IEEE

Authorized licensed use limited to: Technische Universitaet Muenchen. Downloaded on June 10,2024 at 11:43:21 UTC from IEEE Xplore. Restrictions apply.

Still, the methods described above don't take modern hardware and data size into account, with only [15] using network data streaming to mitigate memory constraints.

On the one hand, [16] proposed to utilize a distributed cloud computing system supporting software agents to battle the low-end consumer hardware. On the other hand, [17] proposed to break the queries down into smaller parallel computation fragments, which reduced Non-Uniform Memory Access and network delays.

Based on the downsides of the discussed methods, a need arose to develop a method of procedural landscape generation that would meet the conditions of scalability, uniformity, and consistency, which would allow artists to efficiently control the terrain generation process on a planetary scale.

III. PROPOSED SOLUTION

A method is proposed that mitigates the outlined shortcomings using the following features:

- a distributed generation system with software agent-oriented cloud computation offloading;
- asynchronous generation handling achieved by utilizing Morsel-Driven Query Execution and data streaming;
- an extendible unified agent interface with arbitrary relational table support.

In the proposed solution, architecture is based on the concept that combines a distributed computing system [18] and the morsel-driven parallelism approach to overcome the limitations of consumer hardware.

The proposed system is based on the client-server architecture, where communication is done through gRPC message exchange [19]. A bi-directional streaming approach is used, which allows the application to receive a large amount of data about generated planetoid terrain. This method enables returning partial data to the client within real-time constraints.

The server-side architecture consists of several physical loosely coupled servers interacting with the database, including a central server listening to client requests and preparing responses [20]. Other servers, running as long as the main one, use background workers to pull generation jobs from the database and execute them. It allows distributing resource-expensive planetoid generation execution on multiple servers in a cloud environment. Moreover, each server is uniquely identified, allowing the system to make and handle server-specific or generic requests.

The background worker is implemented based on morsel-driven query execution [17]. Morsel-driven query execution is a concept that takes all advantages of multi-core systems. It allows determining at the runtime what each thread will work on. Morsel-driven parallelism introduces such concepts as a dispatcher (background worker) and execution pipelines (in other terms – planetoid generation jobs). The dispatcher controls and assigns tasks to worker threads. Threads can be moved to other pipelines upon finishing the current job, meaning that no worker stays idle as long as there are jobs to run.

Information about the planetoid generation job includes:

- the connected agent's unique identifier;
- the corresponding server's unique identifier;
- spatial index of the discretized planetoid tile.

It should be noted that the client is agnostic to the spatial tile index used by the server and sends requests based on the geographical coordinate system (Fig. 1). After receiving such a request, the server needs to perform a mapping to the internal coordinate system to store the job, and the generation results in the database. A discrete version of the Quadrilateralized Spherical Cube Projection [21] is chosen to unify the data access patterns for geospatial DB. This projection has a more uniform distortion distribution compared to classical Mercator variants, allowing us to generate rather precise data around the poles.

The planetoid generation jobs execution plan follows the next steps:

1. Background worker pulls jobs from the database considering their priorities and dependencies.

2. Background worker creates separate threads for each pulled job and runs them asynchronously.

3. Each job is associated with a concrete agent and can internally perform some operations in parallel, creating new threads.

4. After the job is completed, the results are saved to the database.

This approach allows for vertical parallelism by executing many unrelated jobs within a background worker and for horizontal parallelism by allowing multithreaded data processing within a job or agent.

Another key concept of the architecture is software agents. A software agent is a separate software module that implements a particular stage of planetoid generation or other information processing. Each agent implements a common interface that allows defining its basic operations and parameters and unifying the agent development process. An agent will specify its dependencies on other software agents and output data types.

By gathering the dependencies of agents in a processing cycle (1), a directed execution dependency [22] graph can be built and validated. Without the graph, some agent dependencies halt later jobs from being executed before prior

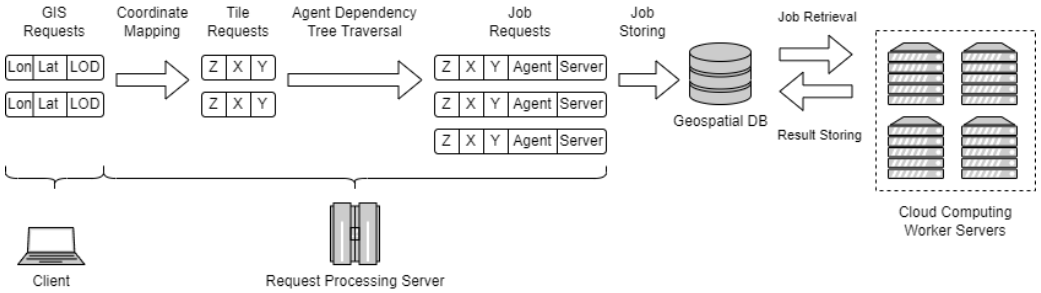


Fig. 1. Generation request processing flow

Authorized licensed use limited to: Technische Universitaet Muenchen. Downloaded on June 10,2024 at 11:43:21 UTC from IEEE Xplore. Restrictions apply.

have finished. But using this feature still allows us to select some of the halted jobs for execution if no more dependent data needs to be generated.

$$r = \sum_i mask(agent_i) \cdot t_i \quad (1)$$

An essential stage in developing a software agent is describing the table schema, which will designate the structure of the output data that would be saved to the database. The proposed approach allows the creation of such tables per software agent at the runtime. As a result, it enables more flexibility in software agent implementation.

IV. RESULTS

The proposed solution is implemented in the PlanetoidGen system. Based on the architecture of the proposed system, MapGen [7] was chosen as the main state-of-the-art competitor in the domain of terrain generation using software agents. Experiments with different input parameters were conducted for the developed version of PlanetoidGen and the provided implementation of MapGen (Table 1).

For benchmarking purposes, only the server side of the PlanetoidGen system was used with only one server instance and one worker using the following runtime: .NET 6.0.6 (6.0.622.26707), X64 RyuJIT. Generation job requests were preset before each benchmark, and the time required for this action was not included in the total time. One job is equivalent to the generation of one tile, where its size is defined by zoom and tile size in pixels.

MapGen is implemented as a C++ console application and does not support distributed computing. It uses the following runtime: Microsoft Visual C++ 2022 X64 (14.32.31326). MapGen agents parameters are set using the command line arguments. One execution is equal to the generation of one tile, where its size is defined as tile size in pixels and kilometers.

Server characteristics: CPU AMD Ryzen 7 5800H (1 CPU, 16 logical and 8 physical cores), GPU Nvidia GeForce RTX 3060, RAM 16GB DDR4, Drive SSD NVME2 2TB, OS Windows 10.0.22000.

Both systems have a raster output – terrain heightmap of a tile. Experiments were conducted for zoom levels 6 (≈156368 km), 7 (≈78184 km), 8 (≈39092 km), and for the following tile sizes in pixels: 128, 256, 512, 1024, and 2048. PlanetoidGen and MapGen have the next set of agents:

- base terrain agent;
- mountain agent;
- hill agent;
- smooth agent;
- shoreline agent.

The time and memory cost comparisons for different tile sizes were recorded (Table 1). In both systems, agent-specific parameters were left as default as they only change the terrain's visual appearance and could be neglected during comparison.

Time and memory costs were recorded in Table 2 to analyze the performance boost for using distributed computing (multiple workers) in the PlanetoidGen system.

The results show that distributed computing in PlanetoidGen implementation provides a significant processing time boost – 8,85 times for 512-pixel tile raster size and 16 workers. Also, even in the example with a single server and single worker, performance was improved compared to MapGen (2,8 times for 512-pixel tile raster size), but generation control was sacrificed, which can be seen on terrain visualizations (Fig 2). Moreover, the runtime and system architecture of PlanetoidGen use additional memory and CPU time for initialization and processing, which is noticeable for small tiles but becomes negligible for higher resolutions.

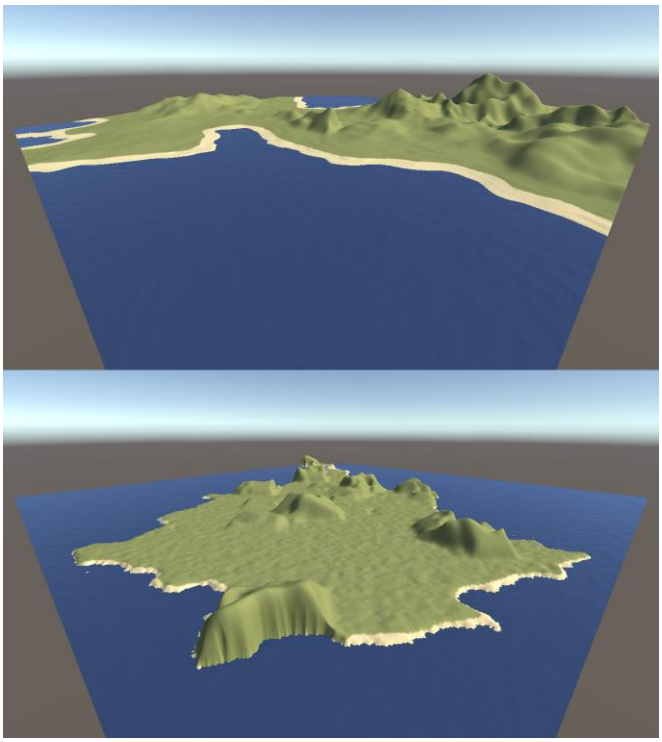


Fig. 2. Terrain visualizations with the same parameters (above – PlanetoidGen, below – MapGen)

CONCLUSION

In conclusion, the concept of a distributed computed system with morsel-driven parallelism was presented, which allows getting the full benefit of a multi-core environment. The proposed architecture combines such solutions as bi-directional gRPC, background workers, execution dependency graphs, and others. In terms of planetoid terrain generation, software agents are used to implement a particular stage of planetoid generation and define their dependencies and outputs. This allows the complex process of generating planetoids to be divided into the implementation of separate agent modules that can be easily integrated or removed from the overall sequence of generation steps.

Authorized licensed use limited to: Technische Universitaet Muenchen. Downloaded on June 10,2024 at 11:43:21 UTC from IEEE Xplore. Restrictions apply.

TABLE I. PERFORMANCE COMPARISON WITH MAPGEN [7]

Zoom	System	Tile raster size									
		128		*256*		*512*		*1024*		*2048*	
		Time, s	***Memory, mb***	***Time, s***	***Memory, mb***	***Time, s***	***Memory, mb***	***Time, s***	***Memory, mb***	***Time, s***	***Memory, mb***
6	MapGen	0.97	0.3749	2.08	1.4117	8.42	5.4754	49.21	21.8096	323.01	86.4342
	PlanetoidGen	0.59	0.5773	1.16	0.9700	2.98	2.5439	11.26	8.8343	47.32	34.0008
7	MapGen	0.97	0.3750	2.09	1.4118	8.32	5.4754	48.94	21.8100	318.56	86.4291
	PlanetoidGen	0.55	0.5854	1.20	0.9700	2.97	2.5438	11.25	8.8431	47.61	34.0095
8	MapGen	0.98	0.3749	2.12	1.4116	8.37	5.4753	49.49	21.8122	327.23	86.4286
	PlanetoidGen	0.55	0.5859	1.16	0.9789	2.98	2.5430	11.02	8.8342	45.86	34.0012

TABLE II. PLANETOIDGEN PERFORMANCE BOOST COMPARISON

Workers	Tile raster size									
	128		*256*		*512*		*1024*		*2048*	
	Time, s	***Memory, mb***	***Time, s***	***Memory, mb***	***Time, s***	***Memory, mb***	***Time, s***	***Memory, mb***	***Time, s***	***Memory, mb***
1	0.55	0.5854	1.20	0.9700	2.97	2.5438	11.25	8.8431	47.61	34.0095
2	0.43	0.5777	0.76	0.9708	1.83	2.5430	6.33	8.8350	26.56	34.0006
4	0.33	0.5771	0.63	0.9707	1.33	2.5433	4.12	8.8348	16.45	34.0005
8	0.30	0.5771	0.57	0.9705	1.08	2.5436	3.20	8.8441	11.74	34.0009

The comparison with MapGen showed the efficiency of the implemented solution in a controlled scenario, being more than 8 times faster for a typical request. Also, in the conducted computational experiments, memory savings of approximately 2 times were achieved.

Future work for this research includes performance and quality improvements, which may help the system for more concrete tasks. Better geographical coordinate projection and geoid usage instead of a planetoid sphere may be used to improve fidelity for the same input. Load balancing, caching, and microservice usage may improve the buffer and delay bound of the system networking, while a different agent selection may allow for more user control.

REFERENCE

[1] V. Cassol et al., “Simulation of Autonomous Agents using Terrain Reasoning,” Proceedings of the 12th IASTED International Conference on Computer Graphics and Imaging, CGIM 2011, Apr. 2011, doi: 10.2316/P.2011.722-017.

[2] T. Hyttinen, E. Mäkinen, and T. Poranen, “Terrain Synthesis Using Noise by Examples,” in Proceedings of the 21st International Academic Mindtrek Conference, 2017, pp. 17–25. doi: 10.1145/3131085.3131099.

[3] K. R. Kamal and Y. S. Uddin, “Parametrically Controlled Terrain Generation,” in Proceedings of the 5th International Conference on Computer Graphics and Interactive Techniques in Australia and Southeast Asia, 2007, pp. 17–23. doi: 10.1145/1321261.1321264.

[4] W. Henham, D. Holloway, and L. Panton, “Broadband acoustic scattering with modern aesthetics from random 3D terrain surfaces generated using the Fourier Synthesis algorithm,” in Acoustics 2016: The Second Australasian Acoustical Societies, Nov. 2016.

[5] R. J. Spick, P. Cowling, and J. A. Walker, “Procedural Generation using Spatial GANs for Region-Specific Learning of Elevation Data,” in 2019 IEEE Conference on Games (CoG), 2019, pp. 1–8. doi: 10.1109/CIG.2019.8848120.

[6] R. J. Spick and J. A. Walker, “Realistic and Textured Terrain Generation Using GANs,” in European Conference on Visual Media Production, 2019, pp. 1–10. doi: 10.1145/3359998.3369407.

[7] J. Doran and I. Parberry, “Controlled Procedural Terrain Generation Using Software Agents,” IEEE Transactions on Computational Intelligence and AI in Games, vol. 2, no. 2, pp. 111–119, 2010, doi: 10.1109/TCIAIG.2010.2049020.

[8] L. O’Boyle, "Agent Based Terrain Generator: Cruthú," M.S. thesis, Grand Valley State Univ., Allendale, Dec. 2018.

[9] Ye. V. Levus, M. Yu. Morozov, R. O. Moravskyi, and P. Ya. Pustelnyk, “LANDSCAPE GENERATION FOR SPHERICAL SURFACES: PROBLEM ANALYSIS AND SOLUTION,” Scientific Bulletin of UNFU, vol. 30, no. 1, pp. 136–141, 2020, doi: 10.36930/40300124.

[10] C. J. Castle and A. T. Crooks, “Principles and concepts of agent-based modelling for developing geospatial simulations,” 2006.

[11] P. Hofmann et al., “Agent Based Image Analysis (ABIA) – Preliminary Research Results from an Implemented Framework,” Sep. 2016. doi: 10.3990/2.455.

[12] M. Jahanbani, M. H. Vahidnia, and M. Aspanani, “Geographical agent-based modeling and satellite image processing with application to facilitate the exploration of minerals in Behshahr, Iran,” Arabian Journal of Geosciences, vol. 15, no. 9, p. 901, May 2022, doi: 10.1007/s12517-022-10165-8.

[13] S. Mohammed, M. Doolan, P. Wernick, and E. Wakelam, “Developing an agent-based simulation model of software evolution,” Information and Software Technology, vol. 96, Nov. 2017, doi: 10.1016/j.infsof.2017.11.013.

[14] O. Aderum and J. Åkerlund, Controllable Procedural Game Map Generation using Software Agents and Mixed Initiative. Malmö högskola/Teknik och samhälle, 2016.

[15] M. Roohitavaf, K. Ren, G. Zhang, and S. Ben-Romdhane, “LogPlayer: Fault-tolerant Exactly-once Delivery using gRPC Asynchronous Streaming,” ArXiv, Nov. 2019, doi: 10.48550/arXiv.1911.11286.

[16] M. Krämer and I. Senner, “A modular software architecture for processing of big geospatial data in the cloud,” Computers & Graphics, vol. 49, Feb. 2015, doi: 10.1016/j.cag.2015.02.005.

[17] V. Leis, P. Boncz, A. Kemper, and T. Neumann, “Morsel-Driven Parallelism: A NUMA-Aware Query Evaluation Framework for the Many-Core Age,” in Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data, 2014, pp. 743–754. doi: 10.1145/2588555.2610507.

[18] R. Jacobsen, J. Jeppesen, K. Laursen, J. Skovsgaard, H. Jensen, T. Toftegaard, “A Scalable Cloud Computing Infrastructure for Geospatial Data Analytics for Change Detection,” Euromicro Conference on Digital System Design, Aug. 2017, pp. 403-410. doi: 10.1109/DSD.2017.49.

[19] E. Jamzuri, H. Mandala, R. Analia, Susanto,. “Cloud-Based Architecture for YOLOv3 Object Detector using gRPC and Protobuf,” Jurnal Teknik Elektro, Jul. 2022, vol. 14, no. 1, pp. 18-23. doi: 10.15294/jte.v14i1.36537.

[20] R. Minailenko, O. Sobinov, O. Konoplitska-Slobodenyuk, K. Buravchenko, “Architectural Features of Distributed Computing Systems,” Central Ukrainian Scientific Bulletin. Technical Sciences, Jan. 2021, vol. 4, no. 35, pp. 16-23, doi: 10.32515/2664-262X.2021.4(35).16-23.

[21] A. Dimitrijevic, M. Lambers, D. Rancic, “Comparison Of Spherical Cube Map Projections Used In Planet-Sized Terrain Rendering,” Mathematics and Informatics of Facta Universitatis, Jan. 2016, pp. 259–297.

[22] M. Han, T. Zhang, Y. Lin, Z. Feng, Q. Deng, Qingxu, “Global Fixed Priority Scheduling with Constructing Execution Dependency in Multiprocessor Real-Time Systems,” Journal of Circuits, Systems and Computers, vol. 27, no. 10, Feb. 2018, doi: 10.1142/S0218126618501657.

Authorized licensed use limited to: Technische Universitaet Muenchen. Downloaded on June 10,2024 at 11:43:21 UTC from IEEE Xplore. Restrictions apply.